PacketVigil: AI Cyberattack Detection Final Project Binder

May 1, 2026

Purpose 3
Definitions 3
Document Definitions 3
Document Acronyms 4
Customer Needs 4
Problem Statement 4
Customer Needs Description 5
High-Level Input-Output (Black Box) Diagram 6
Key Stakeholders 6
General Constraints 6
External Factors — Global, Cultural, and Environmental 6
Social Factors — Public Health, Safety, and Welfare 7
Ethical Factors — Global, Societal, Economic, and Environmental 7
Use Case 8
Product Requirements 9
Target Specifications 11
Standards and Statutory Requirements 12
Benchmarking Information 13
References 13
1. Introduction 14
2. System Architecture 14
2.1 High-Level Architecture Diagram (Updated) 15
2.2 Subsystem Descriptions 16
2.3 Dataflow Walkthrough — Following One Packet Through Figure II-1 16
2.4 Why This Architecture (Design-Decision Rationale) 18
3. Physical / Mechanical Design 18
4. Electrical / Electronic Design 18
5. Software / Firmware Design 19
5.1 Network Ingestion Module 19
5.2 Feature Extraction & Preprocessing Module 19
5.3 ML Model 1: Autoencoder (As-Built) 20
Pseudocode — Sparse Denoising Autoencoder Training 20
Pseudocode — Trained Autoencoder Evaluation 21
5.4 ML Model 2: Random Forest Classifier (As-Built) 21
Pseudocode — Train and Evaluate Random Forest 21

5.5 Sequential Decision Logic (As-Built) 22
5.6 Alerting and Visualization Module (As-Built) 22
6. Module-Level Design 22
7. Data Handling & Interfaces 23
7.1 Data Source and Storage 23
7.2 Internal Data Pipeline 23
7.3 Internal Model Interface 24
7.4 External Interfaces 24
8. Security & Safety Considerations 24
8.1 Electrical Safety 24
8.2 Cybersecurity 24
8.3 Data Integrity 25
8.4 Standards Compliance 25
9. Error Handling & Recovery 25
9.1 Fault Detection 25
9.2 Error Response 25
9.3 Recovery and Resilience 25
9.4 Resilience Testing 26
10. Verification & Validation Plan 27
10.1 Specifications 27
10.2 Test Cases 27
10.3 Data Collection Legend 29
11. Dependency List 29
11.1 Core Environment 29
11.2 Machine Learning Libraries 29
11.3 Network and Data Tools 29
11.4 Dashboard and Logging 29
11.5 Datasets 30
11.6 Platform 30
12. Design Doc Appendices 30
Appendix A. Glossary 30
Appendix B. Pseudocode Reference 30
Appendix C. References (Datasheets, Standards, Technical Papers) 30
A.Testing Results 31
1A. Offline Testing 31
1B. Online Testing 33
4.1 Project Performance, Challenges, and Lessons Learned 34
4.2 Customer Satisfaction and Deliverable Status. 34
4.3 Future Work 34
V. Appendices 35

I. Final Problem Analysis

This section is a revision of the team's original Requirements Specifications document (RS-01). It is updated to reflect what the actual system became as it transitioned from being a concept to a final product. The objective and customer needs have not changed. Where requirements have changed in scope or measurement, the change is called out explicitly. Requirements specified in the original RS-01 but not delivered are marked as Deferred.

Purpose

This document defines the customer needs, research findings, and design requirements for the project AI Cyberattack Detection (PacketVigil). It translates the customer's problem, which is detecting cyberattacks in real time by analyzing network anomalies, into specific functional and engineering requirements with measurable targets. These requirements cover model selection, data processing, performance metrics, and the integration of machine learning with live network monitoring. The document also enumerates constraints, outlines use cases, and specifies verification methods to ensure the system meets the customer's expectations. As an updated specification, it serves as the reference against which the final deliverable was built and verified.

Definitions

Document Definitions

Term	Definition
Design History File	A compilation of records containing the complete design history of a finished device or service.
Verification	Confirmation by testing, analysis, demonstration, and/or inspection that specified requirements have been fulfilled.
Validation	Establishing objective evidence that system specifications conform to user needs and intended uses in the operational environment.
Functional Testing	Testing that ignores the internal mechanism of a system or component and focuses on the outputs generated in response to selected inputs.
Intrusion Detection System (IDS)	A monitoring system that analyzes network traffic for suspicious activity and flags potential intrusions.
Cyberattack	A malicious attempt to disrupt, damage, or gain unauthorized access to computer systems.
Anomaly Detection	Identifying unusual patterns in network data that deviate from a learned baseline of normal behavior.
Signature-Based Detection	Attack detection method relying on predefined patterns of known threats.
False Positive	A network event incorrectly flagged as malicious when it is actually benign.
Autoencoder (AE)	An unsupervised neural network that learns to compress and reconstruct its input; used here for anomaly detection on benign traffic.

Term	Definition
Random Forest (RF)	A supervised ensemble of decision trees that classifies inputs by weighted voting; used here for attack confirmation.
Packet-level feature vector	A 115-dimensional behavioral feature vector produced for every packet by the AfterImage incremental statistics module.

Table 01-1. Document Definitions.

Document Acronyms

Acronym	Description
AE	Autoencoder
AI	Artificial Intelligence
CR	Customer Requirement
DDoS	Distributed Denial of Service
DoS	Denial of Service
FE	Feature Extraction
GIL	Python Global Interpreter Lock
IDS	Intrusion Detection System
IEEE	Institute of Electrical and Electronics Engineers
ML	Machine Learning
MSE	Mean-Squared Error
NIST SP	NIST Special Publication
PCAP	Packet Capture (file format)
pps	Packets Per Second
REST	Representational State Transfer (API style)
RF	Random Forest
SR	System Requirement

Table 01-2. Document Acronyms.

Customer Needs

Problem Statement

The customer needs an effective way to detect cyberattacks in real time. Modern networks face threats including Distributed Denial of Service, data exfiltration, reconnaissance scans, man-in-the-middle attacks, and malware command-and-control traffic. Current signature-based intrusion detection systems struggle with high false-positive rates and a limited ability to recognize novel or evolving attacks because they rely on known patterns. Most open-source ML-based detectors that could fill this gap operate exclusively on

offline, pre-captured datasets, and the few that do operate online tend to require dedicated server hardware. The customer therefore requires a real-time, hybrid AI-based IDS that monitors live network traffic, finds behavioral anomalies, confirms them with a supervised classifier, runs on commodity hardware, and is open enough to be extended or audited.

Customer Needs Description

Modern networks face increasingly sophisticated cyberattacks that are difficult to detect using traditional rule-based intrusion detection systems. Customers — including researchers, educators, and network administrators — need an intelligent system that can analyze large volumes of network traffic and identify unusual patterns in real time. Such a system would provide early warnings of DDoS attacks, data exfiltration attempts, reconnaissance, and malware communications, which often go unnoticed until significant damage occurs.

The customer requires a solution that leverages artificial intelligence and machine learning to go beyond static signature-based detection. The system must adapt to new and evolving threats by learning patterns of normal behavior and flagging anomalies. It must be accurate, fast, and reliable, while presenting results in an understandable way so that both technical and non-technical users can act quickly to mitigate risks.

Ultimately, the project addresses the need for proactive, real-time cyber-defense capabilities in academic and operational settings. By providing an AI-driven intrusion detection framework that runs on commodity hardware, the system empowers users to improve network resilience, safeguard sensitive data, and build a foundation for further research into intelligent cybersecurity solutions.

Need ID	Customer Need	Priority	Final Status
N1	Detect unusual network traffic patterns (DDoS, malware, data exfiltration, reconnaissance, etc.)	High	Met
N2	Provide real-time monitoring and alerting on a live network interface	High	Met
N3	Achieve detection accuracy sufficient to outperform signature-based IDS, with low false-positive rates	High	Met (100% precision on 6/7 datasets)
N4	Offer intuitive dashboards and reports for visualizing alerts and system health	Medium	Met (Flask + Dash)
N5	Be deployable on typical lab or personal computers without specialized or high-cost hardware	Medium	Met (3-core VM, no GPU)
N6	Allow adaptability to evolving threats through machine-learning models that can be retrained	High	Met (open-source pipeline; retraining scripts versioned)

Table 01-3. Customer Needs and final delivery status.

High-Level Input-Output (Black Box) Diagram

Figure 01-1 below shows the system as the customer sees it: live network traffic enters the AI Detection black box and three classes of output emerge — real-time alerts to the administrator, dashboard reports for investigation, and stored logs for forensic analysis.

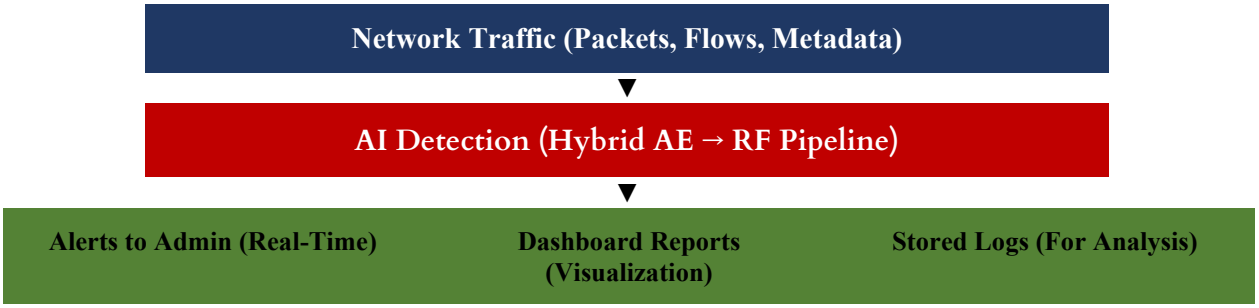


Figure 01-1. High-Level Input-Output (Black Box) Diagram.

Key Stakeholders

- **Customer (External, VT):** Dr. Christiana Garcia — defines the problem, accepts the deliverable, and uses the system to study live network attack detection in an academic setting.
- **Mentor (Internal, VT):** Dr. Joe Adams — provides technical guidance on system architecture, ML methodology, and capstone deliverables throughout development.
- **Graduate Researchers (Internal, VT):** Use the IDS for follow-on experiments, validating its effectiveness in academic research and as a starting point for advanced detection methods.
- **IT Staff / Security Teams (External):** Operate the deployed system to detect and respond to suspicious activity on real networks; care about precision, recall, and runtime stability.
- **Future Developers (Internal/External):** Student teams or open-source contributors who extend the system with new attack profiles, alternative ML stages, or adaptive retraining.

General Constraints

External Factors — Global, Cultural, and Environmental

PacketVigil is intended for use across many environments, from a personal laptop to a small lab network. Because cyberattacks frequently originate across borders and vary by region and threat actor, the system must adapt to traffic mixes that differ from those seen during training. The detector ingests live data streams and can be retrained on captured benign data from the deployment network, allowing it to track changes in normal-traffic baselines over time (→ FUN-1, Data Ingestion; TEST-1, Dataset Evaluation).

Cultural and regulatory factors influence the design as well. Different regions enforce different privacy rules on how network data is collected and stored. To support these constraints, PacketVigil never

stores packet payloads — only running statistics — which limits the privacy surface, and it follows NIST SP 800-94 guidance on the structure of IDS event records so alerts are interpretable across organizations (→ COM-1, NIST Compliance; I/O-2, Standard Outputs).

Environmental impact is also a consideration. Training and inference for ML-based detection can consume significant compute resources. PacketVigil keeps both bounded: the AE and RF are small models that run on CPU without GPU acceleration, the AE filters the vast majority of benign traffic so the RF only evaluates a small fraction of packets, and the pipeline runs comfortably on a 3-core VM. System logs include CPU, memory, and throughput metrics so operators can track efficiency in deployment (→ ENV-1, Resource Metrics Logging).

Social Factors — Public Health, Safety, and Welfare

Cybersecurity directly affects public welfare. A network breach can expose personal information, disrupt healthcare or critical infrastructure, and cost organizations time and money to remediate. PacketVigil supports public welfare by improving attack-detection accuracy and reducing the time between first evidence and an actionable alert, both of which limit the window in which an attacker can cause harm (→ FUN-4, Detection Output; PER-2, Precision).

Performance also affects safety in the operational sense. Too many false positives waste analyst time and lead to ignored alerts ("alert fatigue"); too many false negatives leave systems vulnerable. The system therefore targets both high precision (fewer false alerts) and high recall (fewer missed attacks), and emits alerts within a hard upper bound of 10 seconds of first evidence (→ PER-2, PER-3, PER-4, PER-5, PER-6).

During development the team followed standard lab safety practices: secure handling of dataset files, version control with isolated virtual environments, and testing in containerized or virtualized networks rather than on shared production infrastructure. This minimized risk to live systems and aligned with engineering safety standards for academic capstone work (→ TEST-2, Real-Time Simulation).

Ethical Factors — Global, Societal, Economic, and Environmental

The IEEE Code of Ethics calls on engineers to protect public safety, health, and welfare; to avoid harm; and to be honest about the limits of their work. These principles apply directly to PacketVigil. The system reports its measured precision, recall, and per-attack performance accurately — including the SYN-DoS recall limitation — rather than overstating its capability (→ COM-2, Transparent Reporting).

Globally, the system must be usable by both large organizations and smaller groups with fewer resources. Running on basic computing hardware while still maintaining high detection accuracy

makes the tool accessible to underfunded labs, small enterprises, and student researchers who could not otherwise afford a dedicated security appliance (→ POW-1, Hardware; FUN-3, Hybrid Model Design). Economically, the system is cost-effective: it is software-only on existing hardware, so the BOM is effectively zero (→ COST-1).

Environmental ethics applies to compute and sustainability. Training and evaluation are performed at modest scale — small models, single VM, no GPU — and resource metrics are logged during operation to support capacity planning and to make the system's environmental footprint visible to operators (→ ENV-1, Resource Metrics Logging). Reducing computation without reducing accuracy is a deliberate design choice consistent with responsible use of energy in machine learning.

As engineers, the team's responsibility is to design a system that is transparent, efficient, and protective of the public. This includes testing across diverse attack scenarios (the seven Kitsune datasets plus CIC-IDS-2017), documenting both strengths and weaknesses, and ensuring the system remains stable under sustained operation (→ PER-1, Network Scalability; REL-1, Uptime).

Use Case

The use case below describes how the customer interacts with the deployed system. It is unchanged in intent from the original RS-01 use case, with steps updated to reflect the as-built behavior.

- **Title:** Real-Time Cyberattack Detection in Network Traffic.
- **Primary Goal:** Monitor network traffic in real time and detect potential cyberattacks with high accuracy and minimal false positives.
- **Actors:** Primary actor — PacketVigil. Supporting actors — network administrator (responds to alerts, tunes thresholds), end users (whose traffic is monitored), external threat actors (generating malicious traffic).
- **Preconditions:** PacketVigil is installed on the monitoring host with capture privileges; the target network interface (or a pcap file in replay mode) is available; the AE, RF, and scaler are pre-trained and loaded from the bundled artifacts; the dashboard process is running.
- **Trigger:** Anomalous network traffic patterns are observed that deviate from the trained benign baseline.
- **Main Success Scenario:** (1) Packets flow through the live capture engine; (2) PacketVigil extracts a 115-dimensional feature vector per packet via AfterImage; (3) the AE scores each vector and forwards anomalous ones to the RF; (4) the RF outputs an attack probability; (5) confirmed attacks are written to the JSON alert log and surfaced on the dashboard with timestamps, src/dst IP, src/dst port, AE score, and RF probability; (6) the administrator investigates and responds.
- **Alternative Flows:** If traffic is flagged as anomalous by the AE but the RF probability falls below the configured threshold, the packet is decided benign and no alert is issued. The

administrator can adjust AE or RF thresholds at runtime via the dashboard or REST endpoint to retune sensitivity without restarting the detector.

- **Exception Flows:** If the capture engine fails to start (missing privileges, interface down), PacketVigil exits cleanly with a diagnostic message. Malformed packets are counted and skipped without halting the pipeline. If the ML process crashes, the parent detector logs the event and stops; no partial alerts are emitted.
- **Postconditions:** Successful — confirmed attacks are persisted in alerts.json with full schema and shown on the dashboard. Unsuccessful — the attack is not detected or is misclassified, leading to a missed detection in the log.
- **Notes / Assumptions:** The system assumes the deployment network's benign traffic is similar enough to the training data that AE thresholds remain meaningful; significantly different networks will need re-training. All testing is performed in closed/virtualized environments per academic-capstone safety practice.

Product Requirements

REQ-ID	Category	Priority	Requirement (As-Built)	Verification	Status
FUN-1	Functional	Shall	The system shall ingest live network traffic from a network interface using a high-performance capture engine (dumpcap on Linux, tshark on Windows) and parse Ethernet, IP/ARP, TCP, and UDP layers in real time, without writing intermediate pcap files.	Test (live demo + pcap replay)	Met
FUN-2	Functional	Shall	The system shall produce, for every captured packet, a 115-dimensional behavioral feature vector via the AfterImage incremental statistics module, covering five traffic dimensions across five exponentially-decayed time windows.	Inspection (vector dim) + Test (replay)	Met
FUN-3	Functional	Shall	The detection model shall be a hybrid pipeline: an unsupervised Autoencoder followed sequentially by a supervised Random Forest classifier, where only AE-flagged packets are forwarded to the RF for final classification.	Inspection (code) + Test (replay)	Met
FUN-4	Functional	Shall	The system shall output alerts to the end user, populated with timestamp, flow ID, src/dst IP, src/dst port, AE reconstruction score, RF probability, both decision thresholds, and final verdict.	Demonstration (dashboard) + Inspection (alerts.json)	Met
FUN-5	Functional	Shall	The system shall include a web dashboard for viewing system status, performance metrics, resource usage, alert rate, and a paginated, sortable, filterable table of recent alerts; thresholds shall be adjustable from the dashboard at runtime.	Demonstration (UI demo)	Met
FUN-6	Functional	Should	The system should log all confirmed attack packets in a structured JSON-lines format suitable for post-incident analysis and replay.	Inspection (alerts.json schema)	Met

REQ-ID	Category	Priority	Requirement (As-Built)	Verification	Status
FUN-7	Functional	Shall	The system shall support deterministic offline replay of pcap files through the same pipeline used for live capture, so test results are reproducible against ground-truth labels.	Test (replay against labeled CSV)	Met
PER-1	Performance	Shall	The system shall capture and process network traffic during sustained load representative of the labeled test datasets without dropping packets in the ingestion stage.	Test (sustained replay)	Met
PER-2	Performance	Shall	Per-attack precision shall be ≥ 95% on the offline benchmark datasets.	Analysis (Kitsune + CICIDS-2017 metrics)	Met (100% on 6/7 datasets)
PER-3	Performance	Shall	Per-attack precision in real-time replay shall be ≥ 85%.	Analysis (online replay metrics)	Met
PER-4	Performance	Shall	The system shall emit alerts within a hard upper bound of 10 seconds of first evidence, with a goal of 1–3 seconds.	Analysis (timing instrumentation)	Met (< 1 ms FE+ML / packet)
PER-5	Performance	Shall	Per-attack recall shall be ≥ 95% on the offline benchmark datasets.	Analysis (Kitsune + CICIDS-2017 metrics)	Partially Met (6/7 ≥ 93%; SYN-DoS 22%)
PER-6	Performance	Shall	Per-attack recall in real-time replay shall be ≥ 80%.	Analysis (online replay metrics)	Met (except SYN-DoS)
PER-7	Performance	Should	End-to-end feature-extraction plus ML inference latency should be under 1 ms per packet on a 3-core commodity VM.	Analysis (built-in profiling)	Met (~0.3–0.7 ms/pkt)
PER-8	Performance	Should	Sustained throughput should be at least 600 packets per second under continuous operation.	Test (sustained replay)	Met (697–1272 pps observed)
POW-1	Power / Hardware	Shall	The system shall run on a basic desktop / laptop computer or commodity VM with pre-trained models, without requiring GPU acceleration at inference.	Test (3-core Ubuntu VM)	Met
POW-2	Power / Hardware	Should	Peak memory usage during sustained operation should remain under 1.5 GB on a 3-core VM.	Inspection (resource logs)	Met (peak ≤ 991 MB)

REQ-ID	Category	Priority	Requirement (As-Built)	Verification	Status
I/O-1	Input / Output	Shall	The system shall accept live packet captures and offline pcap files as input, and produce a 115-dimensional feature vector per packet as the standard internal representation.	Test (live + replay)	Met
I/O-2	Input / Output	Shall	The system shall output detections to (a) a JSON-lines log file and (b) the live dashboard, both consuming the same alert schema.	Inspection (alerts.json + UI)	Met
I/O-3	Input / Output	Shall	The system shall expose a REST API for system status, live metrics, alerts, health, and threshold updates.	Test (curl endpoints)	Met
REL-1	Reliability	Shall	The detector shall remain operational under continuous replay long enough to exhaust each labeled test pcap (typically tens of minutes to several hours) without crashing.	Test (long-form replays)	Met
TEST-1	Testing	Shall	Models shall be trained, tested, and benchmarked on existing labeled datasets (CICIDS-2017 plus the seven Kitsune attack scenarios).	Test (offline metrics on each dataset)	Met
TEST-2	Testing	Shall	The final system shall be tested in real-time replay on the chosen attack datasets to validate the live pipeline against offline ground truth.	Test (paced PCAP replay)	Met
COM-1	Compliance	Shall	The alert schema and logging structure shall be consistent with NIST SP 800-94 guidance for IDS event records (timestamp, addresses, ports, protocol, attack indication).	Inspection (side-by-side review)	Met
ENV-1	Environmental	Should	System logs should include resource metrics (CPU usage, memory, throughput) for sustainability and capacity planning.	Inspection (metrics endpoint)	Met
CON-1	Configuration	Deferred	The system shall support role-based access control over logs and configuration.	Demonstration (login flow)	Deferred — not in scope of the final deliverable; called out as future work in the IEEE article and project poster.
COST-1	Cost	Should	Total system cost should remain under $500.	Inspection (BOM)	Met (≈ $0: software-only on existing hardware)
SCH-1	Schedule	Shall	Design approved at end of Semester 1.	Inspection	Met

REQ-ID	Category	Priority	Requirement (As-Built)	Verification	Status
SCH-2	Schedule	Shall	Working application accepted for Student Expo by April 2026.	Inspection (Expo demo)	Met

Table 01-4. Updated System Requirements Matrix (revision of original RS-01).

Target Specifications

The target specifications below are an updated version of the original RS-01 target specifications, with the marginal and ideal targets retained from the original document and an added column reporting the actual measured value from the final build. Detailed numerical results per attack profile are reported in Section III of the binder (Build & Test).

Req ID	Metric	Units	Marginal	Target (Ideal)	Achieved	Traceability / Rationale
PER-2	Detection Precision (Benchmark)	%	≥ 90%	≥ 95%	100% on 6/7 datasets	SYN-DoS at 100% precision but very low recall (see PER-5)
PER-3	Detection Precision (Real-Time)	%	≥ 80%	≥ 90%	≥ 99% on tested datasets	Replay snapshots match offline results
PER-3	False Positive Rate (Both)	%	≤ 10%	≤ 5%	< 1% on 6/7 datasets	AE acts as high-recall filter; RF tightens precision
PER-4	Processing Latency (per pkt)	ms	≤ 3000	≤ 1000	0.3–0.7 ms / pkt	Well below the 10-second hard upper bound
PER-5	Recall (Benchmark)	%	≥ 90%	≥ 95%	≥ 93% on 6/7 datasets	SYN-DoS recall 22% — known limitation
PER-6	Recall (Real-Time)	%	≥ 80%	≥ 85%	≥ 93% on 6/7 datasets	Same caveat for SYN-DoS as PER-5
PER-8	Throughput	pps	≥ 600	≥ 1000	697–1272 pps	Per-dataset variation reflects packet density
POW-2	Peak Memory (3-core VM)	MB	≤ 1500	≤ 1000	572–991 MB	Sustained over full replays

Table 01-5. Updated Target Specifications with achieved values from the final build.

Assumptions:

- Datasets: training and evaluation use publicly available intrusion-detection datasets (CIC-IDS-2017 and the seven Kitsune attack scenarios) as proxies for real-world traffic.

- Baseline: published open-source IDS accuracy benchmarks sit in the 85–90% range; the system's per-attack metrics aim to meet or exceed this on the chosen profiles.
- Network environment: live testing was performed in a virtualized testbed; specifications assume eventual real-world deployment.
- Resource availability: the 3-core Ubuntu VM is taken as a representative "commodity hardware" target; deployments with more cores will see headroom but the targets do not depend on it.

Standards and Statutory Requirements

Req. #	Requirement	Source Document	Details
COM-1	NIST SP 800-94 §2.3.2 (Anomaly-Based Detection)	NIST SP 800-94, Guide to IDPS	Explains benefits/drawbacks of anomaly-based detection; informs the AE training and threshold-tuning approach.
COM-1	NIST SP 800-94 §3.3.11 (Architecture Design)	NIST SP 800-94, Guide to IDPS	Outlines IDS data-processing structure and external interfaces; reflected in the four-subsystem architecture (Section II.2).
COM-1	NIST SP 800-94 §3.3.13 (Securing IDPS Components)	NIST SP 800-94, Guide to IDPS	Recommends role-based access to logs and sensitive components. Targeted in original RS-01 as CON-1; deferred in final build (see Status column in Table 01-4).
COM-1	NIST SP 800-94 §9.3 (Performance Requirements)	NIST SP 800-94, Guide to IDPS	Defines accuracy, precision, and lag-time measurements for IDS evaluation; used as the basis for the Section III metrics.

Table 01-6. Standards and Statutory Requirements.

Benchmarking Information

Competitive benchmarking compares PacketVigil's measured performance against the targets and against representative alternatives. Because the system is a software detector that runs on commodity hardware, the comparison is against (a) the original RS-01 target requirement, (b) reference open-source IDS performance levels considered industry-acceptable, (c) a 'decent' tier, and (d) a 'poor' tier.

Feature / Requirement	Target Requirement	Existing Alternative #1	Existing Alternative #2	Existing Alternative #3
Precision (PER-2 / PER-3)	≥ 95% benchmark; ≥ 85% real-time	90–95% benchmark; 80–90% real-time	85–90% / ~80%	< 85% / < 80%

Feature / Requirement	Target Requirement	Existing Alternative #1	Existing Alternative #2	Existing Alternative #3
Recall (PER-5 / PER-6)	≥ 95% benchmark; ≥ 85% real-time	90–95% / 80–85%	85–90% / ~80%	< 85% / < 80%
False Alarm Rate	≤ 5% benchmark; ≤ 15% real-time	5–10% / 10–15%	10–15% / 15–20%	> 15% / > 20%
Latency (PER-4)	≤ 3 s	3–10 s	10–20 s	> 20 s
Adaptability	Detects most novel attacks via AE	Detects some novel attacks	Detects very few novel attacks	Cannot detect novel attacks
Hardware / Deployment	Consumer PCs, laptops, small VMs	Commodity PCs / servers	Higher-end (GPUs, appliances)	Proprietary cloud / enterprise appliance
PacketVigil Achieved	100% precision and ≥ 93% recall on 6/7 datasets; ≤ 0.7 ms latency; runs on 3-core VM	—	—	—

Table 01-7. Benchmarking Information.

References

[1] C. H. N., "Network Intrusion Dataset (CIC-IDS-2017)," Kaggle, 2017. [Online]. Available: https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset

[2] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. IEEE Symp. Comput. Intell. Secur. Defense Appl., Ottawa, ON, Canada, Jul. 2009, pp. 1–6.

[3] K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," NIST SP 800-94, Feb. 2007.

[4] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in Proc. NDSS Symp., San Diego, CA, USA, Feb. 2018, doi: 10.14722/ndss.2018.23211.

II.Final Detailed Design

This section is the updated Detailed Design Document for the final deliverable, structured per the VT MDE Detailed Design Document template. It is a revision of the team's original detailed design document, updated to reflect what the system actually became as it moved from concept to final build. PacketVigil is a software-only project, so the Physical/Mechanical Design and Electrical/Electronic Design sections of the template are not applicable and are omitted; all other sections of the template are present. Specifications

are documented in Section 10.1 and Test Cases in Section 10.2 (the V&V Plan), as required by the final project binder assignment.

1. Introduction

PacketVigil is a hybrid AI-based intrusion detection system that detects malicious network activity in real time using an unsupervised Autoencoder for anomaly detection followed by a supervised Random Forest classifier for attack confirmation. The system addresses the gap between signature-based IDS — which miss novel attacks — and offline-only ML detectors that cannot run live. PacketVigil runs on commodity hardware, processes packets at packet-level granularity, and surfaces alerts to operators through a web dashboard.

Scope. This document describes the as-built design of the full pipeline: live packet capture, incremental feature extraction, two-stage ML inference, decision logic, alert formatting, and the web dashboard. References to specific source files use repository-relative paths.

Reference documents: (a) the updated Requirements Specification in Section I of this binder, which defines every requirement traced to in this document, and (b) the original RS-01 (30 Sep 2025) and DDD (Nov 2, 2025) for change history. The use case from the requirements specification (Section I, Use Case) is reproduced in this document by reference.

Metric	Target Requirement Description (Updated)
Precision	Offline: ≥ 95% — Achieved on 6/7 datasets. Online: ≥ 85% — Achieved.
Recall	Offline: ≥ 95% — Achieved on 6/7 datasets (SYN-DoS exception, 22%). Online: ≥ 80% — Achieved on tested attacks.
False Positive Rate	≤ 10% — Achieved (< 1% on 6/7 datasets).
Detection Latency	1–3 s goal, 10 s hard upper bound — Achieved (0.3–0.7 ms FE+ML per packet).

Table II-1. Updated performance targets summary.

2. System Architecture

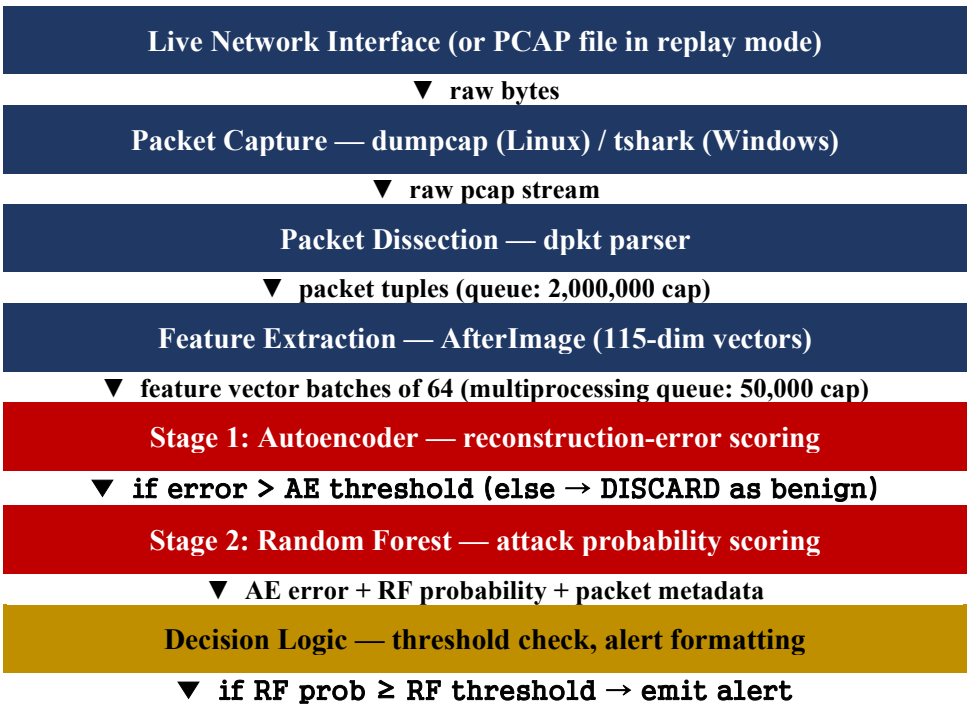
The PacketVigil architecture is best characterized as a hybrid Pipe-and-Filter / Layered architecture with concurrent (multi-process, multi-threaded) execution. Packets flow unidirectionally through a series of independent processing stages (the "filters"), each consuming the output of the previous stage from a bounded queue (the "pipes"). This style was chosen for three reasons that map directly to the project's hardest requirements. First, each filter has a single, well-defined responsibility (capture, parse, extract, score, classify, alert), which keeps any individual component small and testable in isolation. Second, the queues between filters provide elastic buffering: a transient burst in upstream throughput does not stall

downstream stages, which is critical for sustaining the ≥ 600 pps throughput target (PER-8) under bursty traffic. Third, because filters communicate only through queues, the most expensive filter (the ML core) can be hoisted into its own OS process to bypass Python's Global Interpreter Lock (GIL) — without that separation, the < 1 ms per-packet latency target (PER-7) would not have been achievable on a 3-core VM.

The system is also layered: the four subsystems sit at progressively higher levels of abstraction (raw bytes → packet headers → behavioral feature vectors → ML decisions → human-facing alerts). Each layer talks only to the layer immediately above or below it, which is what makes it possible to swap implementations within a layer (e.g., dumpcap on Linux vs. tshark on Windows for capture, or replay-mode vs. live-mode for ingestion) without touching the rest of the system. Figure II-1 below shows the high-level topology of these layers and filters; Sections 2.1 through 2.4 walk through each component in detail.

2.1 High-Level Architecture Diagram (Updated)

Figure II-1 shows the as-built data path through the system. Each labeled box is a filter (an independent processing stage); the arrows are the queues / function calls connecting them. The diagram replaces the original DDD Figure 1, which had drawn the Autoencoder and Random Forest as parallel branches feeding a generic "decision logic" block. The final build is strictly sequential — the AE acts as a high-recall filter that the RF only sees if the AE has already flagged the packet as anomalous.



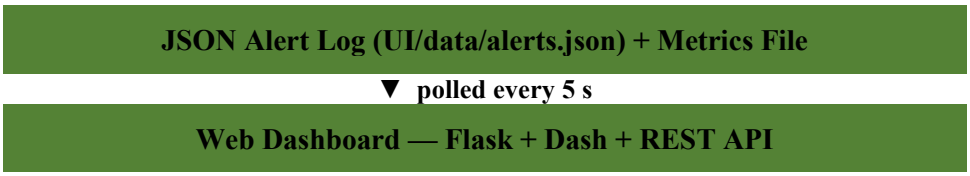


Figure II-1. PacketVigil High-Level Architecture (Pipe-and-Filter, Layered, Concurrent). Navy boxes are ingestion / FE filters, red boxes are the ML core, amber is the decision filter, and green boxes are the output / presentation layer.

2.2 Subsystem Descriptions

Figure II-1 is composed of four logical subsystems, each spanning one or more filter boxes. Table II-2 below names each subsystem and groups the filters in Figure II-1 that belong to it. The right-most column maps each subsystem back to the specific boxes shown in the figure, so the diagram and the table can be read together.

Subsystem	Purpose	Inputs	Outputs	Notes	Maps to Figure II-1 boxes
1. Network Ingestion & Feature Extraction	Captures live packets, parses headers, and produces 115-dim behavioral feature vectors at packet granularity.	Live network interface (or pcap file in replay mode)	115-dim float32 feature vectors, batched 64 at a time	Implemented with dumpcap/tshark + dpkt + AfterImage. No packet payloads are stored — only running statistics.	Boxes 1–4 (Network Interface, Packet Capture, Packet Dissection, Feature Extraction)
2. Hybrid ML Core (AE → RF)	Runs anomaly detection on every feature vector; runs attack classification only on AE-flagged vectors. Implements the strict sequential cascade.	Batches of feature vectors via multiprocessing queue	Per-packet AE error, RF probability (when computed), final verdict	AE in PyTorch, RF in scikit-learn. Runs in its own OS process to bypass the GIL.	Boxes 5–6 (Stage 1: Autoencoder, Stage 2: Random Forest)
3. Decision Logic & Alert Formatting	Applies AE and RF thresholds, formats confirmed attacks into the UI alert schema, and writes them to the alert log.	Per-packet ML results + packet metadata	JSON-lines alert log entries written to disk	OnlineSequential AERF class + ui_bridge module. Default verdict is benign — the AE filters, the RF confirms.	Box 7 (Decision Logic) + Box 8 (JSON Alert Log)
4. Web Dashboard (Flask + Dash)	Presents live system status, metrics, resource usage, and alerts to the operator; exposes a REST API for programmatic access.	Alert log + metrics file written by the detector subsystem	Live UI + REST endpoints (/api/metrics, /api/alerts, /api/threshold, /api/health)	5-second refresh interval. Threshold updates take effect on subsequent packets without restarting the detector.	Box 9 (Web Dashboard); also reads from Box 8 on a 5-second cycle

Table II-2. Subsystem descriptions, with explicit cross-reference to the filter boxes shown in Figure II-1.

2.3 Dataflow Walkthrough — Following One Packet Through Figure II-1

To make the architecture concrete, this section walks through the life of a single packet as it passes through each box in Figure II-1. Numbered steps correspond to the boxes in the diagram (top to bottom).

Step 1 (Network Interface). The packet arrives on the monitored interface, identical to how it would arrive at any other process listening to the wire. In live mode this is a real interface (e.g. veth_detect on the test VM); in offline mode it is a pcap file being replayed at original timing. This first box is part of Subsystem 1 (Network Ingestion & FE).

Step 2 (Packet Capture). dumpcap (or tshark on Windows) captures the packet and writes it to its standard output as part of a raw pcap byte stream. dumpcap is preferred because it has no per-packet parsing overhead — it just hands raw bytes to the next stage. The detector reads this stream directly via a subprocess pipe, eliminating disk I/O. This box is the entry point of Subsystem 1.

Step 3 (Packet Dissection). A reader thread inside the main detector process consumes the raw pcap stream with the dpkt library, parsing the Ethernet header, then IPv4 or ARP, then TCP or UDP. Source / destination IPs, ports, protocol, packet length, and the arrival timestamp are extracted into a lightweight tuple, which is pushed onto a bounded queue. The bounded queue is the first "pipe" in the pipe-and-filter chain — it provides backpressure and lets the parser run faster than the downstream feature extractor without losing packets.

Step 4 (Feature Extraction). A second thread pulls packet tuples off the queue and feeds them into AfterImage, the Cython-accelerated incremental statistics module borrowed from Kitsune. AfterImage maintains hash tables of running statistics keyed by source host, source-destination host pair, source IP/port to destination IP/port channel, full socket, and source-jitter; for each, it tracks five exponentially-decayed time windows ($\lambda \in \{5, 3, 1, 0.1, 0.01\}$). The result is a 115-dimensional float32 feature vector. The FE thread batches these vectors into groups of 64 and places each batch on a multiprocessing queue (cap 50,000) that crosses the process boundary into the ML subsystem.

Step 5 (Stage 1: Autoencoder). The ML process consumes a batch from the multiprocessing queue. For each vector, the persisted MinMaxScaler (scaler.pkl) scales the vector to [0, 1], the trained AE reconstructs it, and the mean-squared reconstruction error is computed. If the error is at or below the configured AE threshold, the packet is decided benign immediately and discarded — it never touches the RF. If the error is above the threshold, the packet is forwarded to Stage 2. This is the load-shedding step that makes the rest of the pipeline real-time: on the SSDP Flood dataset, for example, the AE filters out roughly 99% of benign packets before they reach the RF.

Step 6 (Stage 2: Random Forest). The RF (loaded once at startup from rf_model.joblib) takes the AE-flagged packet's raw (un-scaled) feature vector, runs it through its bundled preprocessor, and outputs a probability that the packet belongs to the attack class. This is a binary classifier trained on labeled benign + attack samples. Both ML stages live inside Subsystem 2.

Step 7 (Decision Logic). The OnlineSequentialAERF class compares the RF probability to the configured RF threshold. If the probability meets or exceeds it, the packet's final verdict is "attack" and an alert is generated; otherwise the verdict is "benign" and no alert is emitted. The class also bumps internal counters (packets seen, AE flags, RF calls, confirmed attacks) that the dashboard later reads. This box is Subsystem 3.

Step 8 (JSON Alert Log). For confirmed attacks, the ui_bridge module formats the result into the UI alert schema (timestamp, flow ID, src/dst IP, src/dst port, AE score, AE threshold, AE verdict, RF probability, RF threshold, RF verdict, final verdict, attack type, ground-truth label) and appends it as one line to UI/data/alerts.json. Writing one JSON object per line is what lets the dashboard tail the file efficiently without re-parsing the whole history every refresh. This is the boundary between Subsystem 3 (which writes) and Subsystem 4 (which reads).

Step 9 (Web Dashboard). The Flask + Dash server polls alerts.json and metrics.json every 5 seconds and updates the dashboard tabs (System Status, Anomaly Inspector, Settings). The same data is also exposed via REST endpoints for programmatic access. This is Subsystem 4 — the final, human-facing layer.

2.4 Why This Architecture (Design-Decision Rationale)

Two design decisions in Figure II-1 deserve explicit justification, because they distinguish this architecture from the one originally proposed in the DDD.

First, the AE and RF are wired in series rather than in parallel. Running them in parallel (the original DDD's approach) would have meant evaluating every packet through both models on every cycle, which is wasteful: the RF is far more expensive per packet than the AE, and the vast majority of traffic on any real network is benign. By using the AE as a high-recall filter and routing only its flagged packets to the RF, the RF's effective load drops by 85–99% on the tested attacks. This is what makes the < 1 ms per-packet latency target achievable on commodity hardware.

Second, the ML core runs in a separate OS process rather than as a thread inside the detector. Python's GIL serializes byte-code execution across threads in a single process, which would force the (compute-heavy) AE forward pass and the (I/O-heavy) capture+FE work to take turns. Splitting them into separate processes lets the OS scheduler put them on different cores, and the multiprocessing queue between them enforces an explicit, measurable handoff that we can profile independently. The cost is the per-packet pickle overhead at the queue boundary, which is why feature vectors are batched into groups of 64 before crossing it.

These two decisions — sequential cascade and process-level parallelism — are what take the architecture from a clean diagram on paper to a system that actually meets PER-4, PER-7, and PER-8 on a 3-core VM.

3. Physical / Mechanical Design

Not applicable. PacketVigil is a software-only project that runs as a process on commodity hardware (a 3-core Ubuntu VM in the validated configuration). There is no enclosure, form factor, or environmental ruggedization to specify. Hardware-related constraints are captured in Section I (POW-1, POW-2).

4. Electrical / Electronic Design

Not applicable. The system has no schematics, power distribution, energy storage, or protection circuits because it is software-only. Power and resource considerations are captured in Section I (POW-1, POW-2, ENV-1) and are verified by the resource-monitoring tests in Section 10.2 below.

5. Software / Firmware Design

Item	Detail
Top-level languages	Python 3.10+. AfterImage statistics module is Cython-accelerated. No C++ was required in the final build.
Capture	dumpcap (Linux, preferred), tcpdump (Linux fallback), tshark (Windows, also used for offline replay).
Packet parsing	dpkt — reads the raw pcap byte stream directly from capture stdout, no intermediate files.
Feature extraction	AfterImage incremental statistics module (Cython-optimized, derived from Kitsune).
ML libraries	PyTorch (Autoencoder); scikit-learn (Random Forest, MinMaxScaler, ColumnTransformer); joblib (model bundle persistence); numpy / pandas.
Web stack	Flask (REST API) + Dash + Dash Bootstrap Components (live UI), all hosted by the same Flask process; flask-cors for cross-origin support during development.
Runtime monitoring	psutil for CPU and memory usage shown on the dashboard.

Table II-3. Top-level languages, dependencies, and APIs in the final build.

5.1 Network Ingestion Module

Field	Detail
Description	Captures live packets (or replays a pcap file), parses headers, and emits per-packet metadata. Replaces the original flow-construction module with a packet-level pipeline that does not store payloads.
Input	Live network interface (dumpcap on Linux, tshark on Windows) or a pcap file in replay mode.
Output	Per-packet tuple: (timestamp, src IP, dst IP, src port, dst port, protocol, packet length) pushed onto a bounded packet queue.
Dependencies	dumpcap / tshark / tcpdump (capture); dpkt (parsing); standard library subprocess and threading.
Data structures	Bounded queue.Queue (max 2,000,000 entries); raw pcap byte stream; packet tuples.
Protocols handled	Ethernet, IPv4, ARP, TCP, UDP.
Exception handling	Malformed packets are counted and skipped; capture process restart raises a runtime error and stops the detector cleanly.

Table II-4. Network Ingestion module specification.

5.2 Feature Extraction & Preprocessing Module

Field	Detail
Description	Maintains incremental statistics over five traffic dimensions and five exponentially-decayed time windows, and emits a 115-dimensional behavioral feature vector for every packet. Replaces the original 'flow-level CSV' design with packet-level vectors.
Input	Per-packet tuples from the Network Ingestion module.
Output	115-dimensional float32 vectors, batched (default 64) onto the ML queue.
Dependencies	AfterImage / netStat (Cython-accelerated); numpy.
Data structures	Hash tables of running statistics keyed by source host, source-destination host pair, source IP/port to destination IP/port channel, full socket, and source-jitter. Entries are pruned to bound memory.
Decay factors (λ)	{5, 3, 1, 0.1, 0.01} — covers short-term burst behavior through long-term flow trends.
Exception handling	Categorical or invalid feature values are clamped or replaced with safe defaults; NaN / inf are replaced before vectors leave the module.

Table II-5. Feature Extraction module specification.

5.3 ML Model 1: Autoencoder (As-Built)

Field	Detail
Description	Symmetric feed-forward denoising autoencoder, trained on benign traffic only. Acts as a high-recall, high-throughput filter: reconstruction error above threshold is forwarded to the RF; otherwise the packet is decided benign and discarded.
Architecture	Encoder: 115 → 64 → 36 → 8. Decoder: 8 → 36 → 64 → 115. ReLU activations and Dropout(p=0.1) in the encoder. Sigmoid on the latent activations to enable a KL-divergence sparsity penalty during training.
Training data	Benign-only network traffic, scaled with a fitted scikit-learn MinMaxScaler that is persisted as scaler.pkl.
Training loss	MSE reconstruction loss + KL sparsity penalty. Sparsity target ρ = 0.05; sparsity weight λ = 1e-3.
Training hyperparameters	Optimizer: Adam, lr = 1e-4. Batch size 1024. Epochs 8. Gaussian noise σ = 0.1 injected on the input as the denoising signal.
Input at inference	Scaled 115-dim float32 vector.
Output at inference	Mean-squared reconstruction error (scalar).
Decision threshold	Tuned per attack profile (e.g. 0.000179 for Fuzzing, 57.347 for SSDP Flood, 2.8 for OS Scan in live deployment). Adjustable at runtime via dashboard / REST.
Exception handling	Feature dimension mismatch raises a ValueError and stops inference; missing scaler / weights fail loudly at startup.

Table II-6. Autoencoder specification (final build).

Pseudocode — Sparse Denoising Autoencoder Training

Algorithm 1 (unchanged from the original DDD): Require: benign network traffic data X; hyperparameters ρ (sparsity target), λ (sparsity weight), σ (noise std), batch size B, learning rate η, epochs E. Ensure: trained autoencoder parameters θ. (1) Normalize X using MinMaxScaler. (2) Form

reconstruction dataset (X, X). (3) Initialize AE with parameters θ. (4) Initialize Adam optimizer with learning rate η. (5) For epoch = 1 to E: for each minibatch x sampled from X: x_noisy ← x + N(0, σ^2); h ← Encoder(x_noisy; θ); $\hat{x}$ ← Decoder(h; θ); L_recon ← MSE($\hat{x}$, x); $\hat{\rho}$ ← mean(h); L_sparse ← Σ ρ·log(ρ/$\hat{\rho}$) + (1−ρ)·log((1−ρ)/(1−$\hat{\rho}$)); L_total ← L_recon + λ·L_sparse; update θ using ∇θ L_total. (6) Save trained model θ and fitted scaler.

Pseudocode — Trained Autoencoder Evaluation

Algorithm 2 (unchanged from the original DDD): Require: preprocessed test CSV, trained model, fitted scaler, threshold τ. Ensure: vector of anomaly predictions for test samples. (1) Load test dataset D_test from CSV. (2) Replace inf and NaN values in D_test. (3) Load saved scaler and transform D_test to obtain X_test. (4) Instantiate AE with same architecture; load trained weights; move to device; set to evaluation mode. (5) With no gradient tracking, for each sample x_i in X_test: compute reconstruction $\hat{x}$_i ← model(x_i); compute reconstruction loss ℓ_i = (1/d)$\|\hat{x}_i - x_i\|^2$; classify as anomaly if ℓ_i > τ. (6) Append y_i to predictions list.

5.4 ML Model 2: Random Forest Classifier (As-Built)

Field	Detail
Description	Supervised binary classifier that confirms or rejects packets flagged as anomalous by the AE. Operates on raw (non-scaled) features through its own preprocessor; bundled with the model in a single joblib artifact.
Architecture	scikit-learn RandomForestClassifier — 400 trees, class_weight='balanced', max_depth=None.
Training data	Labeled benign + attack samples from the CICIDS-2017 and Kitsune datasets, split 80/20 chronologically.
Preprocessing	OneHotEncoder for categorical columns; passthrough for numeric. Stored alongside the trained model in rf_model.joblib.
Leakage controls	Known-leaky features (raw IPs, ports, timestamps) are dropped before training so the RF must rely on behavioral features.
Input at inference	Raw 115-dim feature vector for any packet flagged by the AE.
Output at inference	`Attack probability` ∈ `[0, 1]`.
Decision threshold	Configurable per attack profile (typical 0.01–0.5); adjustable at runtime via dashboard / REST.
Metrics produced offline	ROC-AUC, PR-AUC, F1, precision, recall, confusion matrix; per-row prediction scores saved to CSV.

Table II-7. Random Forest specification (final build).

Pseudocode — Train and Evaluate Random Forest

Algorithm 3 (unchanged from the original DDD): Require: preprocessed dataset with binary labels, preselected threshold τ. Ensure: trained model, evaluation metrics, per-row prediction scores. (1) Load dataset from CSV. (2) If binary task: set y ← 'label' column (0 = benign, 1 = attack); drop 'label' and 'attack'

from feature set. (3) Drop known leaky features (IPs, ports, timestamps). (4) X ← all remaining feature columns. (5) Split chronologically: first 80% → train, last 20% → test. (6) Identify categorical (C) and numeric (N) columns. (7) Build preprocessing pipeline: OneHotEncode C, passthrough N. (8) Initialize RF: 400 trees, class_weight='balanced', max_depth=None. (9) Fit preprocessor on X_train; transform X_train, X_test; train model. (10) Save model bundle: {preprocessor, model, column info}. (11) Predict probabilities $\hat{p} \leftarrow$ model(X_test); class labels $\hat{y}_i = 1[\hat{p}_i \geq \tau]$. (12) Compute and save metrics (JSON) and per-row scores (CSV).

5.5 Sequential Decision Logic (As-Built)

The original DDD left the decision logic block open between three options: cascade, latent-as-input, or weighted voting. The final build commits to the cascade design (also called filter-then-classify):

(1) Compute AE reconstruction error $e(x) = \|x - \hat{x}\|^2 / d$ on the scaled feature vector x.

(2) If $e(x) \leq$ AE threshold τ_{AE}, decide benign and stop.

(3) Otherwise, compute RF attack probability p(x) on the raw feature vector.

(4) If $p(x) \geq$ RF threshold τ_{RF}, decide attack and emit an alert; else decide benign.

(5) The default verdict is benign — the AE's job is to keep the RF's workload small enough to remain real-time, not to make the final call.

This logic is implemented in the OnlineSequentialAERF class (scripts/Hybrid/online_sequential_pipeline.py). It exposes a single per-packet method that takes a raw 115-dim feature vector, applies the saved scaler, runs the AE, conditionally runs the RF, and returns a structured result containing the AE error, RF probability (when computed), final verdict, and packet metadata. The class also maintains running counters (packets seen, AE flags, RF calls, confirmed attacks) that the dashboard reads to display system health.

5.6 Alerting and Visualization Module (As-Built)

- **Alert schema (one JSON object per line in UI/data/alerts.json):** timestamp (ISO 8601 UTC), flow_id, src_ip, dst_ip, src_port, dst_port, ae_score, ae_threshold, ae_verdict, rf_prob, rf_threshold, rf_verdict, final_verdict, attack_type (when known), true_label (when in offline replay).
- **Dashboard tabs:** Dashboard (system status, metrics, alert count and rate, resource usage, active thresholds, uptime); Anomaly Inspector (paginated, filterable, sortable alert table); Settings (live threshold tuning).
- **Refresh:** 5-second interval timer (configurable via DASHBOARD_REFRESH_INTERVAL in api/config.py).
- **REST API:** /api/detection/status, /api/metrics, /api/alerts, /api/health, POST /api/threshold (returns 200 on accept; new thresholds take effect on subsequent packets without restart).

6. Module-Level Design

Each subsystem is implemented as a focused Python module with a small public surface. The list below maps every module in the final build to the requirements it supports.

Module / File	Responsibility	Supports
NetworkIngestion/live_detector_linux.py	Top-level live-capture detector. Spawns dumpcap, runs the reader+FE thread, owns the ML subprocess, writes alerts via ui_bridge.	FUN-1, FUN-3, FUN-4, PER-1, PER-7, PER-8
NetworkIngestion/replay_detector.py	Top-level offline-replay detector. Same pipeline as live detector, reading from a pcap via tshark; produces deterministic results for benchmarking.	FUN-7, TEST-1, TEST-2
NetworkIngestion/FeatureExtractor.py	Wraps AfterImage; produces a 115-dim feature vector per packet.	FUN-2
NetworkIngestion/netStat.py	Cython-accelerated AfterImage incremental statistics.	FUN-2, PER-7
NetworkIngestion/ui_bridge.py	Formats per-packet results into the UI alert schema and appends to alerts.json; clears the file at start of run.	FUN-4, FUN-6, I/O-2
scripts/autoencoder/Autoencoder.py	Defines the symmetric AE (115→64→36→8); used at training and inference.	FUN-3
scripts/autoencoder/AE_config.py	Single source of truth for AE architecture, training hyperparameters, and persistence paths.	FUN-3
scripts/autoencoder/AE_train.py	Training loop: scaler fit, denoising MSE + KL sparsity, weight save.	TEST-1
scripts/Hybrid/online_sequential_pipeline.py	OnlineSequentialAERF class — sequential AE → RF decision logic, per-packet API.	FUN-3, PER-4, PER-7
scripts/Hybrid/Hybrid_Sequential_AE_RF.py	Offline batch evaluator for the same sequential logic; produces precision / recall / F1 / confusion matrices.	TEST-1, PER-2, PER-5
UI/api/app.py	Flask + Dash server bootstrap.	FUN-5, I/O-3
UI/api/api_routes.py	REST endpoints (/api/metrics, /api/alerts, /api/threshold, /api/health).	I/O-3
UI/api/dash_layout.py / dash_callbacks.py	Live dashboard layout and reactive callbacks.	FUN-5
UI/api/data_manager.py	Reads alerts.json + metrics.json; backs both the API and the UI with the same view.	FUN-4, ENV-1

Table II-8. Module-to-requirement traceability for the final build.

7. Data Handling & Interfaces

7.1 Data Source and Storage

Training and evaluation use publicly available labeled datasets containing benign and attack flows: CIC-IDS-2017 for an initial benchmark, and the seven Kitsune attack scenarios (Fuzzing, Active Wiretap, SYN-DoS, SSDP Flood, OS Scan, ARP MitM, Mirai Botnet) as the primary evaluation suite. Each dataset comes as a pcap file plus a labeled CSV. Pcaps are used both for offline metric computation and for live-traffic

simulation via paced replay. Datasets are stored locally for experimentation and on the team GitHub repository in versioned form so that any test can be reproduced from a clean checkout.

7.2 Internal Data Pipeline

The internal data pipeline is the same in live mode and replay mode. Raw pcap bytes from dumpcap or tshark flow into dpkt, which produces per-packet tuples (Step 3 in Section 2.3). AfterImage consumes those tuples and produces 115-dimensional float32 feature vectors (Step 4). Vectors are batched into groups of 64 and pushed across a multiprocessing queue into the ML process. The AE consumes scaled vectors and emits a reconstruction-error scalar per packet. The RF consumes raw (un-scaled) vectors for AE-flagged packets and emits an attack probability.

7.3 Internal Model Interface

- AE input: scaled 115-dim float32 vector (MinMaxScaler from scaler.pkl).
- AE output: mean-squared reconstruction error (scalar, float32).
- RF input: raw 115-dim float32 vector (no scaling); only computed for AE-flagged packets.
- RF output: attack probability $\in [0, 1]$ (float32).
- Decision-logic output: structured dict containing AE error, RF probability (when computed), final verdict (benign / attack), and packet metadata.

7.4 External Interfaces

- **Capture interface:** subprocess pipe to dumpcap or tshark, reading raw pcap bytes from stdout. No intermediate disk files.
- **Alert log:** UI/data/alerts.json — JSON-lines, one alert per line, schema documented in Section 5.6.
- **Metrics file:** UI/data/metrics.json — running performance and resource counters, polled by the dashboard.
- **REST API:** /api/detection/status, /api/metrics, /api/alerts, /api/health, POST /api/threshold. Default port 5000.
- **Dashboard UI:** browser-based, served by the same Flask process as the API on http://<host>:5000/dashboard/.

8. Security & Safety Considerations

8.1 Electrical Safety

Not applicable in any new way. PacketVigil runs as a software process on a standard computer; electrical safety, grounding, and power protection are handled by the host system and its power supply. The design does not modify or interface directly with hardware components, so electrical hazards are minimal.

8.2 Cybersecurity

PacketVigil reads network traffic that the host can already observe; it never generates or modifies packets. The primary security concerns are therefore protecting the model artifacts, alert log, and dashboard from tampering by an attacker who has gained host access.

- **Model integrity:** AE weights, scaler, and RF bundle are versioned in the repository so any tampering can be detected by re-checkout. Loading mismatched artifacts (e.g. a 115-dim scaler against a different model) fails loudly at startup with a ValueError.
- **Input handling:** All packet input is treated as untrusted. Malformed packets are counted and skipped without halting the pipeline; NaN/inf values are replaced before reaching the AE.
- **Privilege separation:** The capture process runs with packet-capture privileges (CAP_NET_RAW on Linux); the ML and dashboard processes do not require elevated privileges.
- **Role-based access (deferred):** The original RS-01 specified user roles via CON-1; this was deferred from the final build (see Status column in Table 01-4) and is called out as future work.

8.3 Data Integrity

Alerts are written as append-only JSON-lines to UI/data/alerts.json. Each line is a complete, self-describing record so partial writes can be detected and discarded. Both alert and metrics files are owned by the user running the detector and are not exposed via the dashboard for modification — the UI is read-only with respect to the alert log.

8.4 Standards Compliance

PacketVigil's alert schema and logging behavior align with NIST SP 800-94 §3.3.11 (Architecture Design) and §9.3 (Performance Requirements). Role-based access (NIST SP 800-94 §3.3.13) was originally targeted via CON-1 in RS-01 but was deferred from the final build to focus engineering effort on the detection pipeline, dashboard, and benchmarking.

9. Error Handling & Recovery

9.1 Fault Detection

The detector continuously monitors its own performance and health and writes the results to UI/data/metrics.json: packets processed, AE flags, RF calls, confirmed attacks, average and peak FE+ML latency, throughput (pps), peak memory, and total CPU time. The dashboard reads this file every 5 seconds and renders a color-coded alert-rate badge (Normal / Elevated / Critical), giving the operator at-a-glance visibility into the system's health.

9.2 Error Response

- **Capture process failure:** If dumpcap / tshark fails to start (missing privileges, bad interface, missing pcap file), the detector raises a RuntimeError with a diagnostic message and exits cleanly. No partial alerts are written.
- **Malformed packets:** Counted in fe_stats['malformed'] and skipped; do not halt the pipeline.
- **Missing model artifacts:** Missing scaler.pkl, autoencoder.pt, or rf_model.joblib raises FileNotFoundError at startup before any packets are processed.

- **Feature dimension mismatch:** Raises ValueError before inference; this catches scaler/model version mismatches early.
- **ML process death:** The parent detector logs the event and stops; it does not attempt to silently restart the ML process, because doing so could mask a real fault.

9.3 Recovery and Resilience

The system is designed for recoverable single-runs rather than long-lived self-healing operation. After a fault, the operator restarts the detector with the same arguments; warm-up (the first 86,890 packets) re-establishes AfterImage's running state before scoring resumes. Because alerts are append-only JSON-lines, restarting after a fault does not corrupt previously-written alerts; the dashboard simply continues reading from the file as new lines arrive.

9.4 Resilience Testing

Resilience is verified in V&V test cases TC-7 (sustained throughput), TC-8 (resource use under load), and TC-10 (long-form replay) — see Section 10.2 below.

10. Verification & Validation Plan

This section combines (1) the system specifications and (2) the test cases that verify the build meets the requirements documented in Section I.B. Specifications are encoded directly in source — AE_config.py for the model, replay_detector.py and live_detector_linux.py for runtime, api/config.py for the UI — and are reproducible from the project repository.

10.1 Specifications

Specification	Value / Description
Feature vector dimension	115 (5 traffic dimensions × 5 time windows × per-window stats), produced by AfterImage.
AE input scaling	scikit-learn MinMaxScaler fitted on the benign training set, persisted as scripts/autoencoder/scaler.pkl.
AE architecture	Encoder 115 → 64 → 36 → 8; decoder 8 → 36 → 64 → 115. ReLU + Dropout(0.1) in encoder; sigmoid on latent for sparsity penalty.
AE training	MSE reconstruction loss + KL sparsity penalty (target 0.05, weight 1e-3); Gaussian noise σ = 0.1; Adam, lr = 1e-4; batch 1024; 8 epochs; benign-only data.
AE decision threshold (live)	Per-attack tuned (e.g. Fuzzing 0.000179, SSDP Flood 57.347, OS Scan 2.8). Adjustable at runtime.
RF model	scikit-learn RandomForestClassifier (400 trees, class_weight='balanced', max_depth=None) loaded from rf_model.joblib (preprocessor + estimator bundle).
RF decision threshold (live)	Configurable per attack profile (e.g. 0.01–0.5); adjustable at runtime via dashboard / REST.
Capture engine	dumpcap on Linux (preferred) with tcpdump fallback; tshark on Windows.
Packet parser	dpkt reading raw pcap stream from capture stdout (no intermediate files).
Pipeline architecture	Reader thread → FE thread → multiprocessing queue → ML process. FE batches of 64 vectors; ML batch size 256.
Queue capacities	Packet queue: 2,000,000; ML queue: 50,000.
Warm-up	86,890 packets are fed through AfterImage before ML scoring begins, so live state matches the trained baseline.
Alert log format	JSON-lines, one alert per line, written by ui_bridge to UI/data/alerts.json.
Dashboard stack	Flask + Dash + Dash Bootstrap Components; REST API on port 5000.
Dashboard refresh	5-second interval timer (configurable via api/config.py).
Target hardware	3–core Ubuntu VM, ≤ 1.5 GB RAM, no GPU at inference.

Table II-9. Final build specifications.

10.2 Test Cases

Each test case below traces back to one or more requirements from Section I.B and is designed so that pass / fail can be determined directly from the test artifacts (offline metrics JSON, replay alert log, dashboard screenshots, or process resource report). Numerical results per attack profile are reported in Section III of this binder (Build & Test).

Test ID	Requirement(s)	Type	Procedure	Pass Criterion
TC-1	FUN-1, FUN-2, FUN-7, I/O-1	Test	Replay each Kitsune pcap (Fuzzing, Active Wiretap, SYN-DoS, SSDP Flood, OS Scan, ARP MitM, Mirai Botnet) end-to-end through the live ingestion pipeline using replay_detector.py.	Pipeline produces a 115-dim vector per packet for every packet in the capture; no exceptions; vector count equals packet count minus malformed packets.
TC-2	FUN-3, PER-5	Analysis	For each replayed dataset, run the AE-only stage with the tuned threshold and compute precision, recall, and F1 against ground-truth labels.	AE recall is high enough to act as a high-recall filter (≥ 98% on six of seven datasets, demonstrating its filter role).
TC-3	FUN-3, PER-2, PER-5	Analysis	Run the full sequential AE → RF pipeline on each replayed dataset and compute final precision, recall, and F1.	Sequential precision ≥ 95% (offline) and recall ≥ 95% (offline) on the target attack profiles. SYN-DoS is reported as a known partial-met case (high precision, low RF recall).
TC-4	FUN-4, FUN-6, I/O-2	Inspection	Verify that every packet for which the RF returns probability ≥ threshold produces exactly one JSON-line entry in alerts.json with the documented schema.	Alert count in alerts.json equals the number of confirmed attacks reported by the detector's internal counters; all required fields populated and valid.
TC-5	FUN-5, I/O-3	Demonstration	Open the dashboard during a live replay, observe metrics refresh, and adjust AE / RF thresholds via the UI (and via POST /api/threshold).	Metrics, alert table, and resource usage update on the configured interval; threshold changes take effect on subsequent packets without a detector restart.
TC-6	PER-4, PER-7	Analysis	Measure end-to-end FE+ML latency per packet during replay using the pipeline's built-in timing instrumentation.	Mean per-packet FE+ML latency < 1 ms; first-evidence-to-alert latency well within the 10-second hard upper bound.
TC-7	PER-1, PER-8	Test	Run a sustained replay long enough to exhaust the input pcap and record the average throughput in packets per second.	Sustained throughput ≥ 600 pps; ≥ 1000 pps on Fuzzing and SSDP Flood. Ingestion stage drops zero packets.
TC-8	POW-1, POW-2, ENV-1	Inspection	Run each replay on the 3-core Ubuntu VM and record peak memory, total CPU time, and CPU capacity used (data shown on dashboard and in metrics.json).	Peak memory ≤ 1.5 GB; CPU capacity used leaves headroom on at least one core for ingestion bursts.
TC-9	TEST-1, TEST-2	Test	From a clean checkout of the repository, retrain the AE, regenerate the scaler, retrain the RF, and re-run TC-2 / TC-3.	Reported metrics match within numerical tolerance, demonstrating that all artifacts needed to reproduce results are versioned.

Test ID	Requirement(s)	Type	Procedure	Pass Criterion
TC-10	REL-1	Test	Run a long-form replay (multi-million-packet pcap, e.g. Fuzzing) to completion under nominal load.	Detector completes the run without crashing; resource use stays within POW-2 limits throughout.
TC-11	COM-1	Inspection	Compare a sample of alert log entries against the alert-record fields recommended by NIST SP 800-94 §3.3.11 / §9.3.	All recommended fields (timestamp, src/dst IP, src/dst port, protocol, attack indication) are present in every alert.

Table II-10. Test cases for the final build, traced to Section I.B requirements.

10.3 Data Collection Legend

The following definitions apply to the metrics computed in TC-2, TC-3, TC-6, and TC-7:

- Precision = TP / (TP + FP)
- Recall = TP / (TP + FN)
- F1 Score = 2 · (Precision · Recall) / (Precision + Recall)
- `Detection Latency = time of decision generation − time of flow arrival from ingestion`
- Throughput (pps) = packets processed / wall-clock seconds

Ground-truth labels for offline metrics come from the Kitsune and CIC-IDS-2017 labeled CSVs. In-pipeline timing is recorded by the detector's built-in instrumentation, which writes metrics.json alongside alerts.json on every run.

11. Dependency List

PacketVigil is software-only, so the deliverable's "bill of materials" is the list of runtime and development dependencies below.

11.1 Core Environment

- Python 3.10+
- Cython (used to accelerate the AfterImage statistics module)

11.2 Machine Learning Libraries

- PyTorch — Autoencoder training and inference
- scikit-learn — Random Forest, MinMaxScaler, ColumnTransformer
- numpy, pandas — data handling and preprocessing
- joblib — model bundle persistence

11.3 Network and Data Tools

- dumpcap (preferred) / tcpdump (fallback) — Linux live capture
- tshark — Windows capture and offline pcap parsing
- dpkt — packet parsing from raw pcap bytes

11.4 Dashboard and Logging

- Flask — REST API
- Dash + Dash Bootstrap Components — live UI
- flask-cors — cross-origin support during development
- psutil — CPU and memory monitoring

11.5 Datasets

- CIC-IDS-2017 — initial offline benchmark
- Kitsune attack scenarios (Fuzzing, Active Wiretap, SYN-DoS, SSDP Flood, OS Scan, ARP MitM, Mirai Botnet) — primary evaluation suite

11.6 Platform

- Ubuntu Linux (validated on a 3-core VM) for live capture
- Windows for offline pcap replay and development

12. Design Doc Appendices

Appendix A. Glossary

See Section I, Document Definitions and Document Acronyms (Tables 01-1 and 01-2).

Appendix B. Pseudocode Reference

Algorithms 1, 2, and 3 (Sparse Denoising Autoencoder Training, Trained Autoencoder Evaluation, and Train/Evaluate Random Forest) are inlined in Sections 5.3 and 5.4 of this document.

Appendix C. References (Datasheets, Standards, Technical Papers)

[1] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in Proc. NDSS Symp., San Diego, CA, USA, Feb. 2018, doi: 10.14722/ndss.2018.23211.

[2] K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," NIST SP 800-94, Feb. 2007.

[3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP), 2018.

[4] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[5] G. E. Hinton and R. R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," Science, vol. 313, no. 5786, pp. 504–507, 2006.

III. Build & Test

A.Testing Results

1A. Offline Testing

Below shows the performance results for our 7 datasets across different attack types. Performance is great for all attacks but SYN-DoS. The RF performs much more poorly than the AE for SYN-DoS, indicating that the RF cannot identify the attack patterns well. This is a great area for future work, which will be discussed in later sections.

Additionally, a few dataset notes are needed. First, every test set was temporally sampled in order to balance the attack-benign data 1-1. The only ones that were not were SYN-DoS and Mirai Botnet due to their exceptionally high amount of attack data, there was no possible temporal sampling split that could balance the benign/attack packets in the test set. For SSDP flood, the false positive number is exceedingly low, meaning that the imbalanced dataset is not inflating statistics.

Dataset	AE Precision	AE Recall	AE F1	RF F1	Sequential Precision	Sequential Recall	Sequential F1	% Passed to RF
Fuzzing	49.0%	98.8%	65.5%	99.4%	100.0%	97.6%	98.8%	Attack passed: **98.80%** Benign passed: **86.82%**
Active Wiretap	60.5%	93.3%	73.4%	83.8%	100.0%	93.5%	96.5%	Attack passed: **93.25%** Benign passed: **60.96%**
SYN-DoS	96.2%	100.0%	98.1%	35%	100.0%	22%	35.5%	Attack passed: **100.00%**

								Benign passed: **0.27%**
SSDP Flood (Imbalanced)	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	Attack passed: **100.00%**
								Benign passed: **0.15%**
OS Scan	51%	98.85%	67%	99.9%	100%	98.85%	99.42%	Attack passed: **98.85%**
								Benign passed: **15.17%**
ARP MitM	49.99%	99.96%	66.65%	100.00%	100.00%	99.96%	99.98%	Attack passed: **99.96%**
								Benign passed: **100.00%**
Mirai Botnet	100%	100%	100%	100%	100%	100%	100%	N/A

Table 3.1: Offline Performance Metrics.

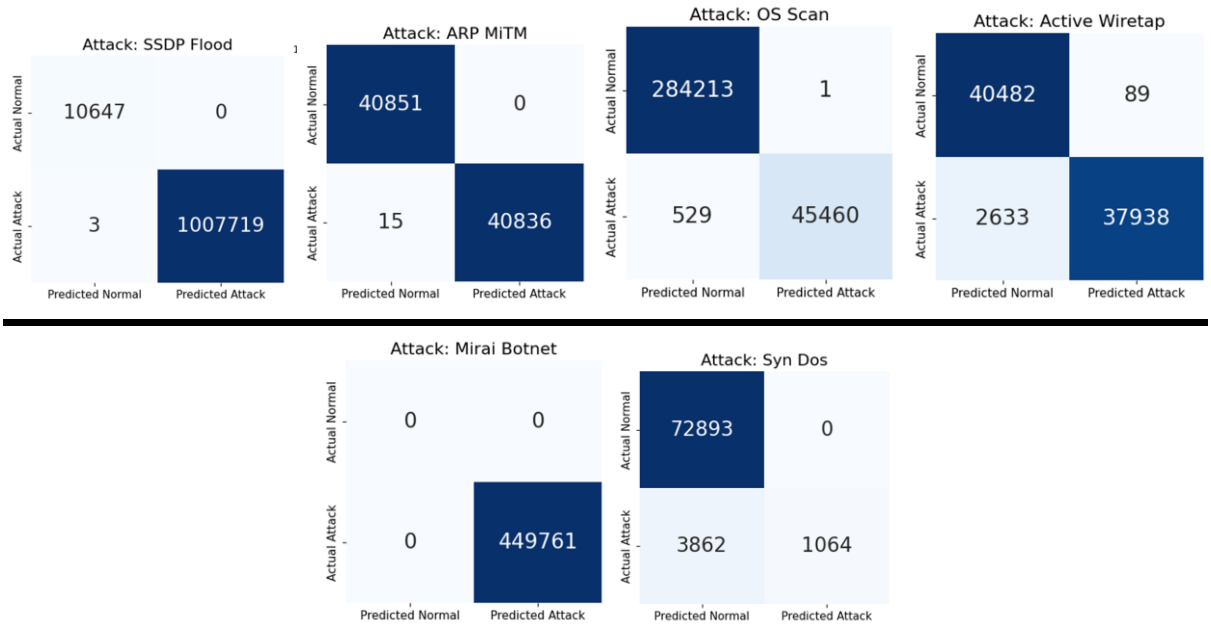


Figure 3.1: Offline Testing Confusion Matrices

1B. Online Testing

Below shows two tables that display the results of three online testing runs across three different attack types: fuzzing, SSDP flood, and OS scan. The first table shows the detection and timings results, while the second table shows the dataset statistics about average packet transmission rate, test size, etc.

Dataset	Precision, Recall, F1	Resource Usage (Ubuntu VM, 3 cores)	Throughput	FE+ML Latency (per pkt)
Fuzzing	**Precision:** 1.0000 **Recall:** 0.9991 **F1 Score:** 0.9995 **TP** 302,671 **FP** 10 **TN** 358,805 **FN**: 277	**Peak Memory:** = 991.3 MB **CPU Capacity Used:** ~22%	1231 pps	~0.7ms
SSDP Flood	**Precision**: 1.00 **Recall:** 0.99 **F1 Score:** 1.00 **TP** 590,011 **FP** 13 **TN** 6,863 **FN**: 3,113	**Peak Memory:** 948.6 MB **CPU Capacity Used:** ~59%	1272 pps	~0.7ms
OS Scan	**Precision**: 1.00 **Recall:** 0.9937 **F1 Score:** 0.9968	**Peak Memory:** 572.3 MB **CPU Capacity Used:** ~8%	697 pps	~0.3 ms

Table 3.2: Online Performance Metrics.

Dataset	Test set Packets	Total Transmission Time	Average pps	True Negative (Benign)	True Positive (Attack)
Fuzzing	661,763 packets	~28 minutes	~1291	358,815 packets	299306 packets
SSDP	600k packets	~30 minutes	~1400, peaks at	6,863 packets	590,011

Flood			25,000		packets
OS Scan	330,204 packets	~10 minutes	712 pps	284,213 packets	45,936 packets

Table 3.1: Online Testing Datasets.

IIII. Delivery

4.1 Project Performance, Challenges, and Lessons Learned

Overall, the project turned out to be very successful. Packetvigil is a fully integrated live pipeline that monitors network data, ingests packets, and flags attacks in real time.

This project presented numerous challenges that the team had to work through. First, was determining the feature extraction mechanism. The team had to determine which algorithms, tools, and approaches needed to be taken in order to ingest network data and extract useful metadata from it without compromising real-time functionality. This required thorough research and repetitive design tweaks and edits. Secondly, devising a replicable, robust online testing approach was also a difficult task. A majority of academic papers limit their benchmarking to offline testing via .csv dataset validation. Our team needed to discover tools and design a safe, online testing sandbox.

Throughout the year, the team learned the importance of planning and scheduling. The team consistently planned out their design approaches, while mapping deliverables to a structured timeline. This kept the team on time and productive. In addition, the team got better at utilizing all the members as time went on. Subteams were created for multiple key design checkpoints to distribute work and boost productivity.

4.2 Customer Satisfaction and Deliverable Status.

The customer, Dr. Garcia, has expressed that she is very much pleased with the project performance. This document, including all code, has been sent to her as well.

4.3 Future Work

Several promising directions exist for future work with PacketVigil. First, the feature extraction engine could be improved to reduce its susceptibility to timing jitter artifacts introduced during traffic replay, which has been shown to distort time-sensitive features and degrade detection accuracy.. Second, alternative decision logic approaches — such as ensemble methods or weighted decision voting— warrant exploration to further improve robustness and adaptability across varying attack types. Third, and most critically, cross-attack generalization testing should be expanded. Initial results here are highly encouraging, as an autoencoder trained exclusively on an OS-scan dataset achieves greater than 95% precision, recall, and F1-score when evaluated on fuzzing traffic, demonstrating that the Autoencoder generalizes superbly across structurally distinct attack classes. This, coupled with more work with decision logic, could greatly propel the effectiveness of the detection engine.

V. Appendices

Source code and files are attached to this document as a folder.